COS 350 Lecture 5

unix: gdb & ddd debuggers

string funtions

case study: strlen()

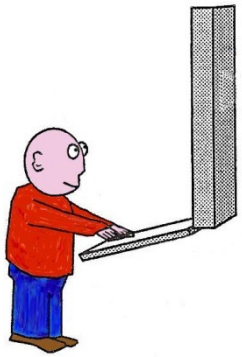
pointer math

case study: strtok()

gdb - the gnu debugger

gcc –g prog.c -o prog	-g compile for debugging
gdb prog	text based debugger
ddd prog	visual interface to gdb

commands	
run *[command line args]*	
break *func name or line number*	in ddd can double click start of line
next	next line (step over)
step	step into function call
finish	finish current function (step out)
cont	continue until reach next breakpoint
print *var*	print value of variable (has formating)
display *variable*	display every step
graph display *variable*	draw in separate window
bt	backtrace (show call stack)
up	move debugger view up call stack
down	move debugger view down call stack
quit	quit debugger



demo ddd on mycat through X-windows

```
#include <string.h>
```

comparison:

```
int strcmp(char *s1, char *s2);                 - string compare
int strncmp(char *s1, char *s2, int n);         - only first n characters
int strcasecmp(char *s1, char *s2);             - case insensitive
int strncasecmp(char *s1, char *s2, int n);     - only first n characters
```

length:

```
int strlen(char *s);                            - length
```

copying:

```
char *strcpy(char *dest, char *src);            - copy dest ← src
char *strncpy(char *s1, char *s2, int n);       - max n characters
char *strcat(char *dest, char *src);            - concatenate dest ← dest+src
char *strncat(char *dest, char *src, int n);    - add at most n char
char *strdup(char *s1);                         - allocate and duplicate
```

searching:

```
char *strchr(char *s, int c);
char *strrchr(char *s, int c);
int strspn(char *s, char *accept);

int strcspn(char *s, char *reject);

char *strpbrk(char *s, char *accept);

char *strstr(char *haystack, char *needle);
```

- *search **s** for char **c***
- *reverse search*
- *length of initial span of **s** of chars from **accept***
- *complement span (chars not in **reject**)*
- *pointer into **s** of first match of any char from **accept***
- *find **needle** in **haystack***

tokenizing:

```
char *strtok(char *s, char *delimiters);
```

- *chop **s** into tokens*

Case study: strlen() using array syntax vs pointer syntax

```
// strlen using array syntax
int strlen_1(char s[]) {
   int len = 0;

   while( s[len] != '\0' )
      len++;

   return len;
}
```

The image part with relationship ID rId3 was not found in the file.

```
// strlen using pointer syntax
int strlen_2(char *s) {
   char *start = s;

   while( *s )
      s++;

   return s - start;
}
```

The image part with relationship ID rId3 was not found in the file.

Which is better?

Case study: strlen() using array syntax vs pointer syntax

```
// strlen using array syntax
int strlen_1(char s[]) {
   int len = 0;

   while( s[len] != '\0' )
      len++;

   return len;
}
```

gcc (on x86 architecture)
7 assembly language instrs per loop iteration

gcc –O2
4 assembly language instrs per loop iteration

```
// strlen using pointer syntax
int strlen_2(char *s) {
   char *start = s;

   while( *s )
      s++;

   return s - start;
}
```

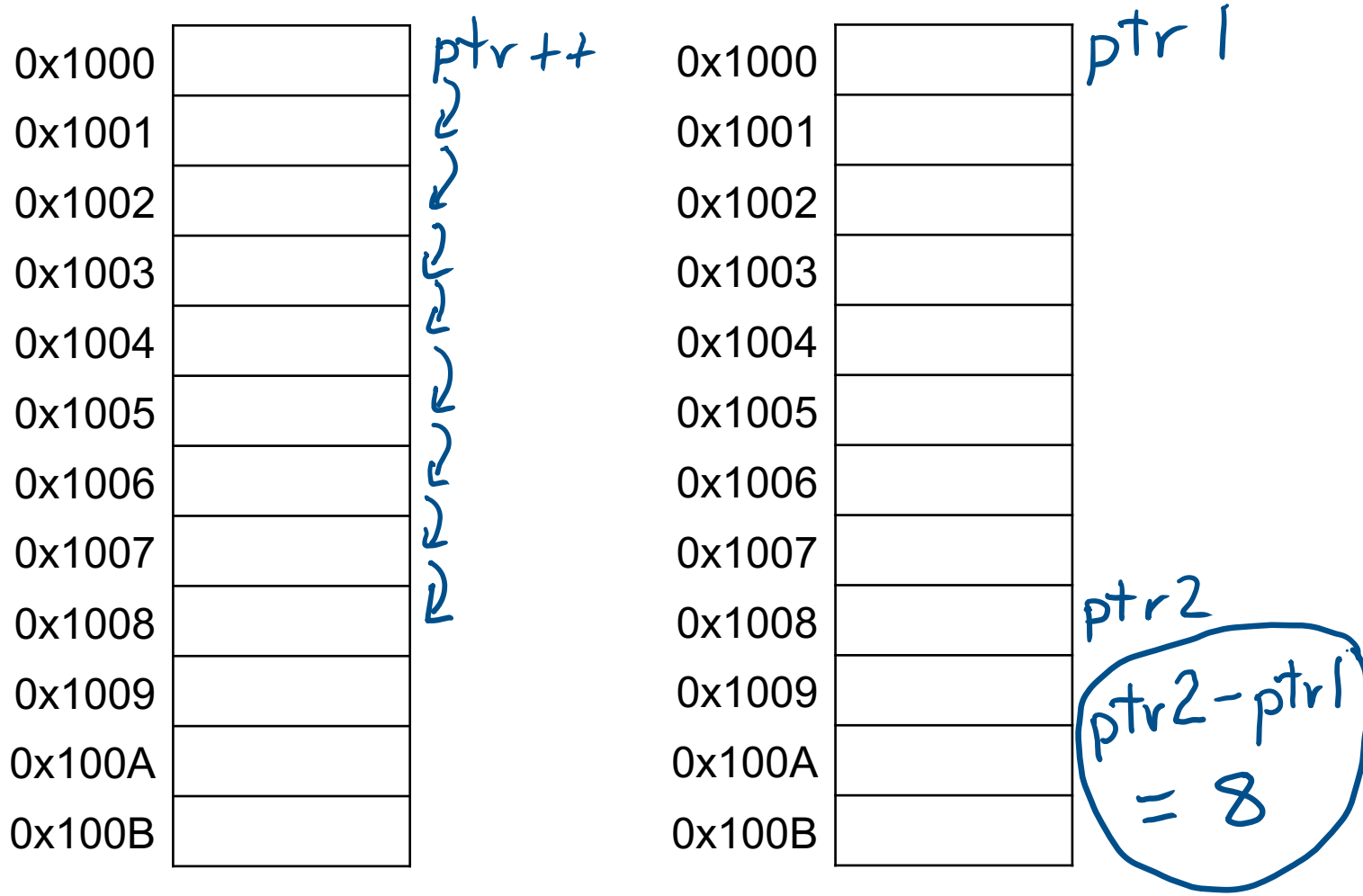
gcc
5 assembly language instrs per loop iteration

gcc –O2
3 assembly language instrs per loop iteration

Pointer math (is intended for working with arrays)

`ptr++`	moves to the next cell (adjusts pointer by the size of the type)
`ptr2 - ptr1`	distance in number of cells
`ptr1 < ptr2`	relative position of pointers

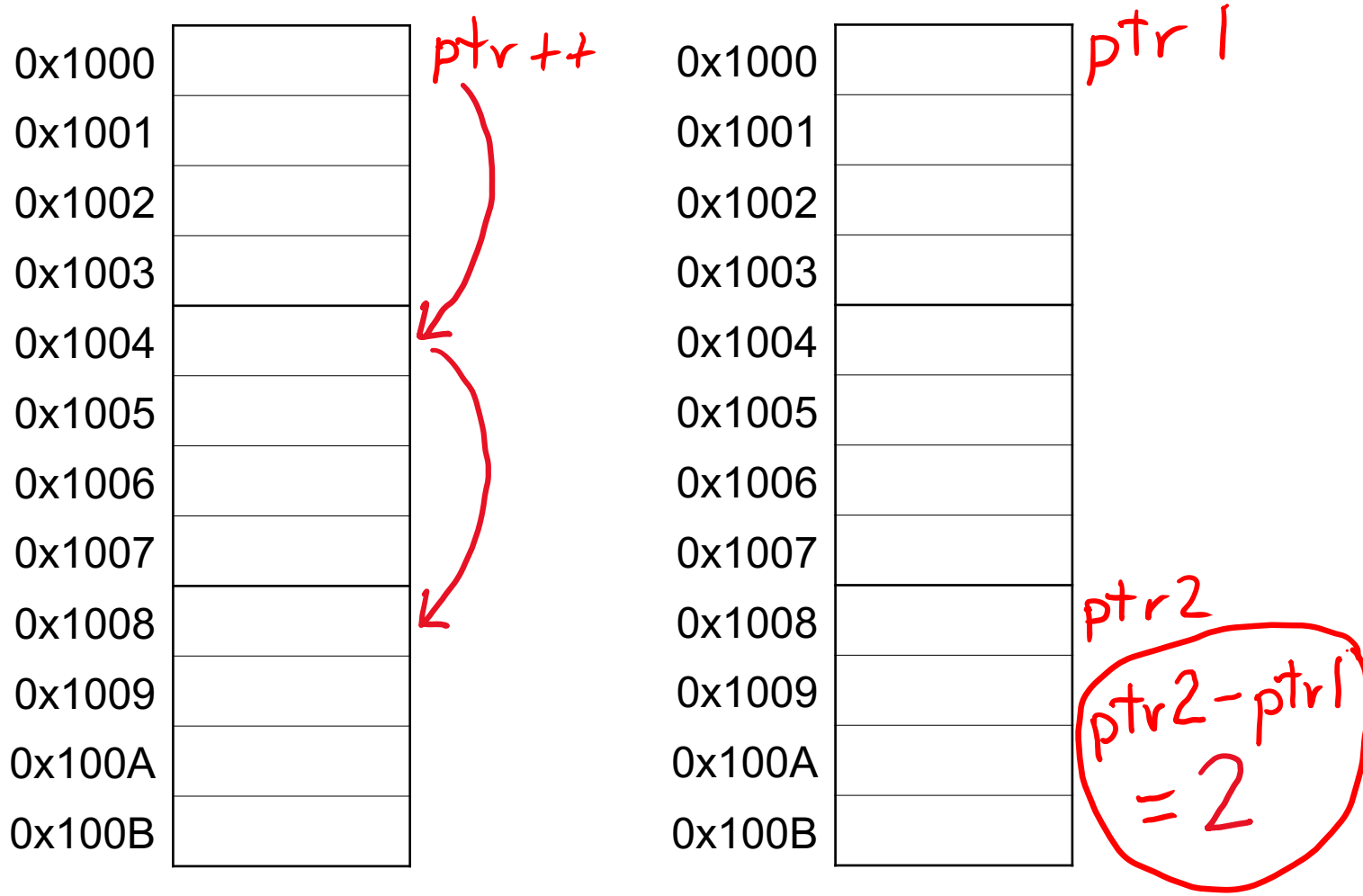
```
char *ptr,
     *ptr1,
     *ptr2;
```



Pointer math (is intended for working with arrays)

`ptr++`	moves to the next cell (adjusts pointer by the size of the type)
`ptr2 - ptr1`	distance in number of cells
`ptr1 < ptr2`	relative position of pointers

```
int *ptr,
    *ptr1,
    *ptr2;
```



Case study: strtok()

first call: strtok(string, delimiters);
remaining calls: strtok(NULL, delimiters);
returns tokens 1 at a time, NULL when done

```
...
printf("buf=\"%s\"\n", buf);
printf("tokens:\n");

char *token = strtok(buf, " ");
while (token) {
   printf("  \"%s\"\n", token);
   token = strtok(NULL, " ");
}
```

```
buf="O bla di, o bla da"
tokens:
  "O"
  "bla"
  "di,"
  "o"
  "bla"
  "da"
  NULL
```

O		b	l	a		d	a	,		o		b	l	a		d	a	Ø

```
// tokenize string - note this modifies the string
// by inserting '\0' chars to break it into tokens
// the first call passes in the string s, and subsequent calls pass s=NULL
char *strtok(char *s, const char *delims) {
  static char *current = NULL;   // remember position between calls

  if (s)
    current = s;                 // passed in a new string

  current += strspn(current, delims); // skip past delimiters

  if (!*current)
    return NULL;                 // end of string, no more tokens

  char *retval = current;        // save start of token

  current += strcspn(current, delims);  // skip until delim

  if (*current)                  // if not yet at end
    *current++ = 0;              // mark end of token, and move past it

  return retval;
}
```